

Prompt techniczny dla agenta Google Antigravity

Skopij poniższą treść bezpośrednio do okna dialogowego agenta w środowisku Google Antigravity:

Kontekst zadania: Projekt zaliczeniowy z przedmiotu „Automatyzacja procesów projektowania z wykorzystaniem Pythona” (dr Stanisław Gepner, Politechnika Warszawska). Projekt stanowi jednocześnie moduł rejestracji odometrii referencyjnej (Ground Truth baseline) do pracy magisterskiej dotyczącej nawigacji robota w środowisku z przeźroczystymi przeszkodami.

Dostępny jest plik z wykładami (Lectures_python_sgepner.pdf) oraz repozytorium przedmiotu.

Cel: Zaimplementowanie ultralekkiego, 2-skryptowego systemu akwizycji, binarnej serializacji oraz wizualizacji trajektorii robota. Kod musi ściśle realizować wzorzec serializacji obiektowej i buforów numerycznych z **Wykładu 5** (klasa danych, msgpack, kodery ExtType, rekonstrukcja przez np.frombuffer) oraz wizualizacji z **Wykładu 3** (matplotlib).

Wymagania architektoniczne i ograniczenia kodu:

- Całość projektu musi zamknąć się w dokładnie **2 skryptach**.
- Objętość każdego skryptu nie może przekraczać **60–70 linii**.
- Rezygnujemy z baz danych (SQLite) i sieciowych gniazd UDP. Dane zapisywane są bezpośrednio do pliku binarnego .dat.

Specyfikacja Skryptu 1: telemetry_recorder.py (Węzeł ROS2 + Rejestrator binarny)

1. **Model danych (Zgodny z Wykładem 2 i 5):**
 - Zdefiniuj klasę TrajectorySession, która w konstruktorze przyjmuje description (tekstowy opis eksperymentu).
 - Klasa przechowuje: znacznik czasu utworzenia (timestamp), opis (description), liczbę próbek (sample_count) oraz współrzędne robota jako listy lub tablice x i y.
2. **Akwizycja danych z ROS2:**
 - Węzeł subskrybuje topic /odom (nav_msgs/msg/Odometry).
 - W funkcji zwrotnej (callback) odczytuje msg.pose.pose.position.x oraz msg.pose.pose.position.y i dopisuje je do wewnętrznych buforów.
3. **Serializacja do pliku (Zgodna z Wykładem 5):**
 - Implementacja funkcji kodującej enc_telemetry(obj), która obsługuje typ TrajectorySession.
 - Współrzędne \$x\$ i \$y\$ muszą zostać przekonwertowane na tablice numpy.ndarray typu float64, a następnie na ciąg bajtów za pomocą metody .tobytes() (uwaga: nie używać przestarzałego tostring()).
 - Funkcja kodująca pakuje atrybuty za pomocą msgpack.ExtType(code=1, data=msgpack.packb(...)).
 - Zrzut do pliku: Zapis następuje po przechwyceniu sygnału przerwania (KeyboardInterrupt / Ctrl+C). Skrypt otwiera plik w trybie binarnym (open('trajectory_data.dat', 'wb')) i wywołuje msgpack.dump(session, f, default=enc_telemetry).

Specyfikacja Skryptu 2: telemetry_plotter.py (Deserializator + Analiza i Wizualizacja)

1. **Środowisko:**

- Skrypt autonomiczny (czysty Python, bez konieczności importowania rclpy).
- 2. **Deserializacja (Zgodna z Wykładem 5):**
 - Implementacja funkcji `dnc_telemetry(code, data)`.
 - Dla `code == 1` funkcja rozpakowuje dane za pomocą `msgpack.unpackb()`, a tablice współrzędnych rekonstruuje z buforów binarnych za pomocą `np.frombuffer(buf, dtype=np.float64)`.
 - Odczyt pliku w trybie binarnym: `msgpack.load(open('trajectory_data.dat', 'rb'), ext_hook=dnc_telemetry)`.
- 3. **Wizualizacja (Zgodna z Wykładem 3):**
 - Wygenerowanie wykresu 2D trajektorii robota $y(x)$ za pomocą `matplotlib.pyplot`.
 - Wykres musi zawierać: siatkę (`plt.grid()`), oznaczenie punktu startu i końca trasy, zachowanie proporcji osi (`plt.axis('equal')`) oraz tytuł zawierający odczytany z pliku opis sesji i datę wykonania pomiaru.

Kryteria akceptacji dla agenta:

1. Kod musi być sformatowany w sposób zwięzły, czytelny i pozbawiony zbędnych bibliotek zewnętrznych poza `rclpy`, `numpy`, `msgpack` oraz `matplotlib`.
2. Formatowanie binarne musi być w 100% kompatybilne pomiędzy skryptem 1 a 2.
3. Podaj instrukcję terminalową do przetestowania całego potoku (uruchomienie `telemetry_recorder.py`, wygenerowanie przykładowego ruchu z topicu `/odom`, przerwanie nagrywania i uruchomienie `telemetry_plotter.py`).

Słownik nowych pojęć i komend

- **`plt.axis('equal')`:** *Plot Axis Equal* – instrukcja biblioteki `Matplotlib` wymuszająca jednakową skalę jednostkową na osiach odciętych (X) i rzędnych (Y). Zapobiega to geometrycznemu zniekształceniu wykreślanej trajektorii ruchu robota.
- **`KeyboardInterrupt`:** Wbudowany wyjątek systemu Python podnoszony w momencie, gdy użytkownik wymusi przerwanie procesu w terminalu kombinacją klawiszy `Ctrl+C`. Służy do bezpiecznego zamknięcia pętli rejestratora i bezbłędnego zrzucenia bufora pamięci do pliku.